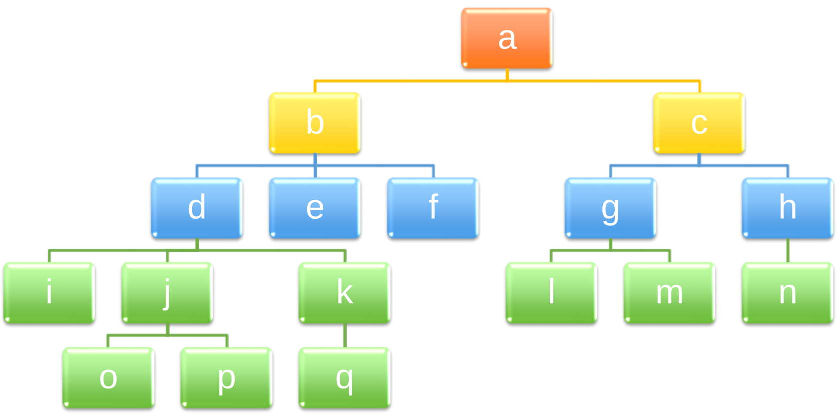
Tree Structures

1

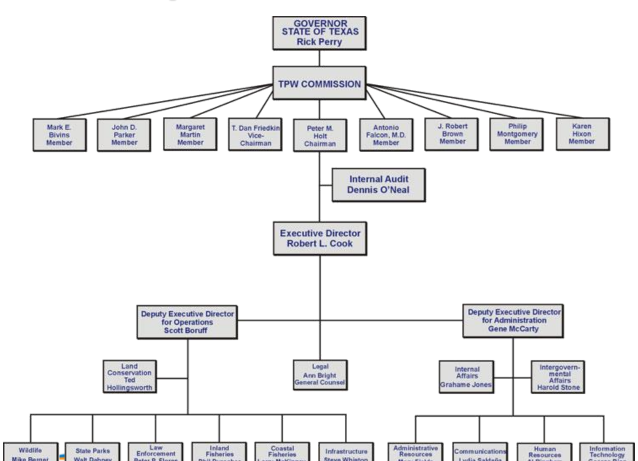
Contents

2

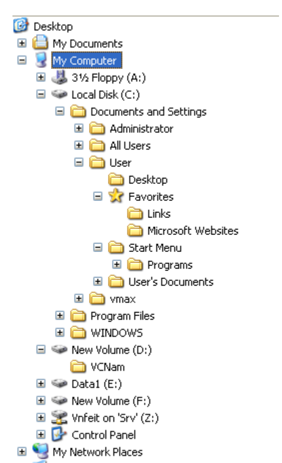
Some Examples



Some Examples



Organizational chart

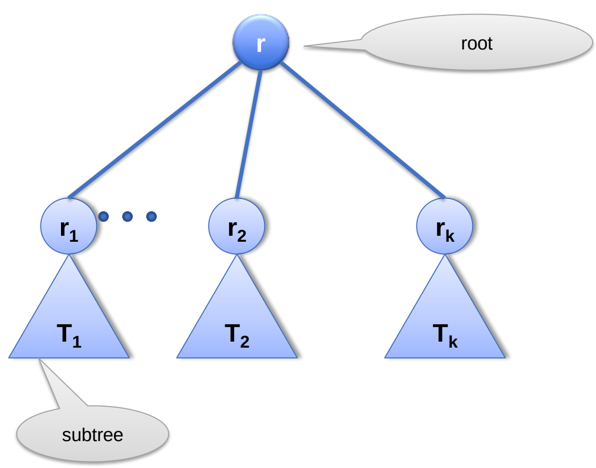


Directory tree

Trees

- Used to represent **relationships**
- ***hierarchical*** in nature
  - “Parent-child” relationship exists between nodes in tree.
  - Generalized to ancestor and descendant
  - Lines between the **nodes** are called **edges**
- A **subtree** in a tree is any node in the tree together with all of its descendants

Trees



fit@hcmus

Terminologies

- node: an item/element in a tree.
- parent (of node *n*): The node **directly above** node *n* in the tree.
- child (of node *n*): The node **directly below** node *n* in the tree.
- root: The only node in the tree with no parent.
- leaf: A node with no children.
- path: A sequence of nodes and edges connecting a node with the nodes below it.

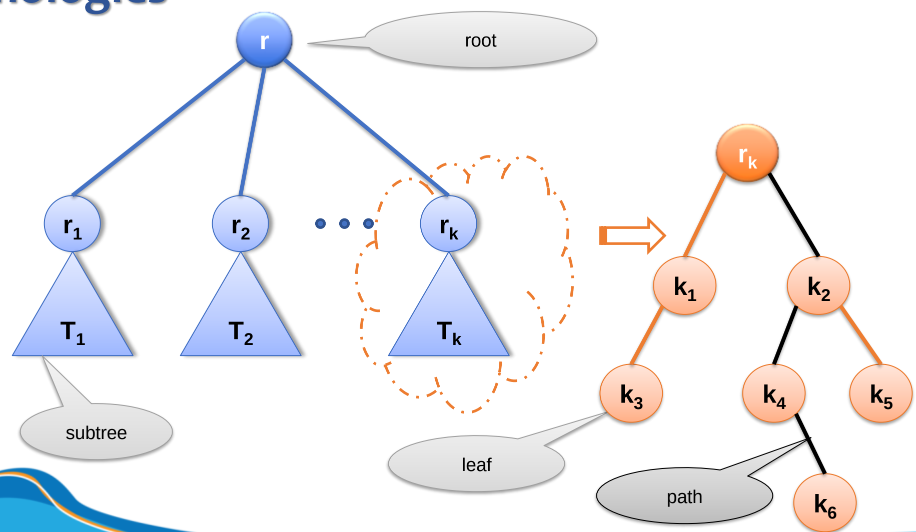
fit@hcmus

Terminologies

- siblings: Nodes with common parent.
- ancestor (of node *n*): a node on the path from the root to *n*.
- descendant (of node *n*): a node on the path from node *n* to a leaf.
- subtree (of node *n*): A tree that consists of a child (if any) of *n* and the child's descendants.

fit@hcmus

Terminologies



fit@hcmus

Terminologies

- degree/order
  - Order of node *n*: number of children of node *n*.
  - Order of a tree: the maximum order of nodes in that tree.
- depth/level (of node *n)*
  - If *n* is the root of *T* , it is at level 1.
  - If *n* is not the root of *T* , its level is 1 greater than the level of its parent.

```
if node n is root:
    level(n) = 1
Otherwise:
   level(n) = 1 + level(parent(n))
```

Terminologies

- Height of tree: number of nodes in the longest path from the root to a leaf.

- Height of a tree T in terms of the levels of its nodes
  - If T is empty, its height is 0.
  - If T is not empty, its height is equal to the maximum level of its nodes.

fit@hcmus

Terminologies

- Height of tree *T*:

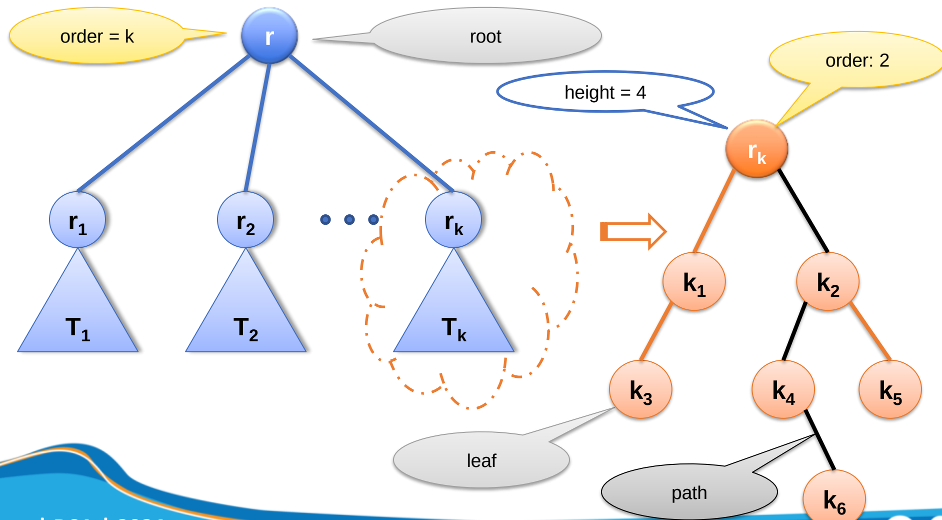
```
if T is empty:
   height(T) = 0
Otherwise:
   height (T) = max{level(Ni)}, Ni∈T
```

- Height of tree *T*:

```
if T is empty:
   height(T) = 0
Otherwise:
   height(T) = 1 + max{height(Ti)}, Ti is a subtree of T
```

fit@hcmus

Terminologies



Kinds of Trees

fit@hcmus

General Tree

- Set T of one or more nodes such that T is partitioned into disjoint subsets
  - A single node r , the root
  - Sets that are general trees, called subtrees of r

fit@hcmus

n-ary Tree

- set T of nodes that is either empty or partitioned into disjoint subsets:
  - A single node r , the root
  - n possibly empty sets that are n-ary subtrees of r

Binary Tree

- Set T of nodes that is either empty or partitioned into disjoint subsets
  - Single node r , the root
  - Two possibly empty sets that are binary trees, called left and right subtrees of r

Traversals

Traversal

- Visit each node in a tree **exactly once**.
- Many operations need using tree traversals.
- The basic tree traversals:
  - Pre-order
  - In-order
  - Post-order

Pre-order Traversal

```
PreOrder(root)
{
   if root is empty
      Do_nothing;
   Visit root; //Print, Add, ...
   //Traverse every Child_i.
   PreOrder(Child_0);
   PreOrder(Child_1);
   …
   PreOrder(Child_k-1);
}
```

Post-order Traversal

fit@hcmus

```
PostOrder(root)
{
  if root is empty
     Do_nothing;
  //Traverse every Child_i
  PostOrder(Child_0);
  PostOrder(Child_1);
  …
  PostOrder(Child_k-1);
  Visit at root; //Print, Add, ...
}
```


21

In-order Traversal

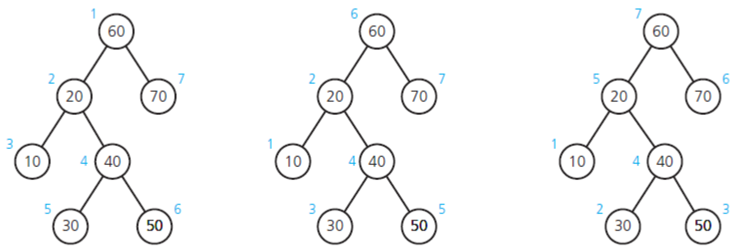
fit@hcmus

```
InOrder(root)
{
  if root is empty
     Do_nothing;
  //Traverse the child at the first position
  InOrder(Child_0);
  Visit at root;
  //Traverse other children
  InOrder(Child_1);
  InOrder(Child_2);
  …
  InOrder(Child_k-1);
}
```


22

fit@hcmus

Traversals



(a) Preorder: 60, 20, 10, 40, 30, 50, 70 (b) Inorder: 10, 20, 30, 40, 50, 60, 70 (c) Postorder: 10, 30, 50, 40, 20, 70, 60

(Numbers beside nodes indicate traversal order.)

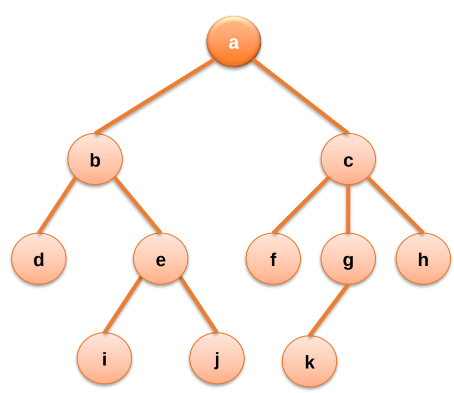
fit@hcmus

Examples

Pre-order

In-order

Post-order



fit@hcmus

Examples

Pre-order

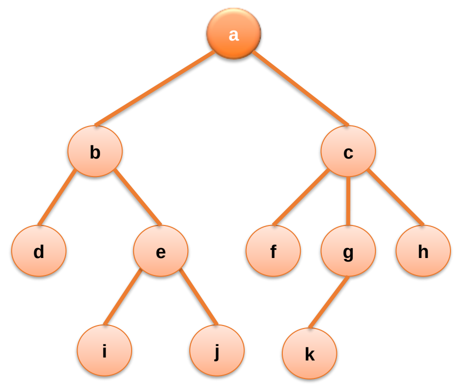
- *a b d e i j c f g k h*

In-order

- d b i e j a f c k g h

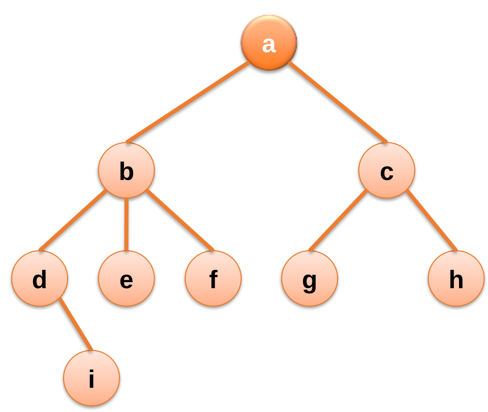
Post-order

- *d i j e b f k g h c a*



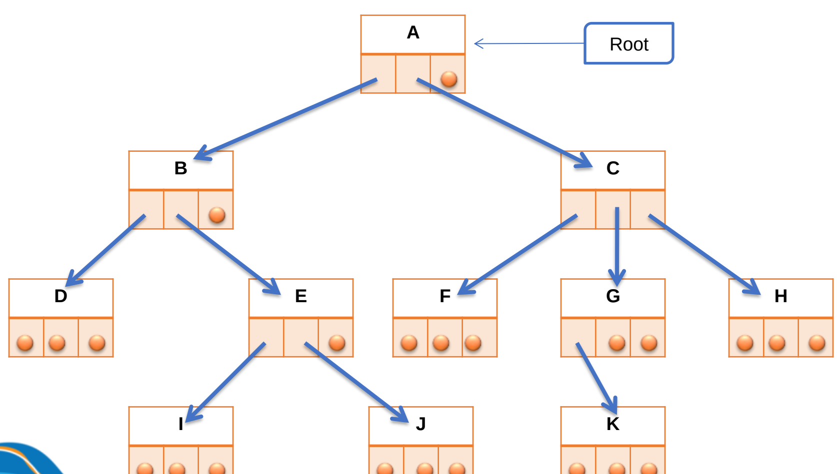
fit@hcmus

Examples



Tree Representation

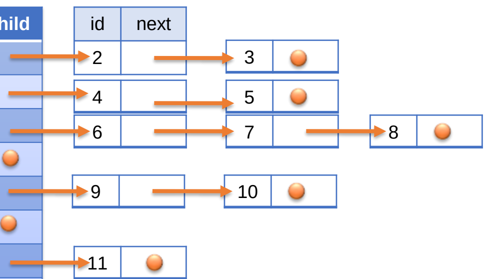
Tree Representation

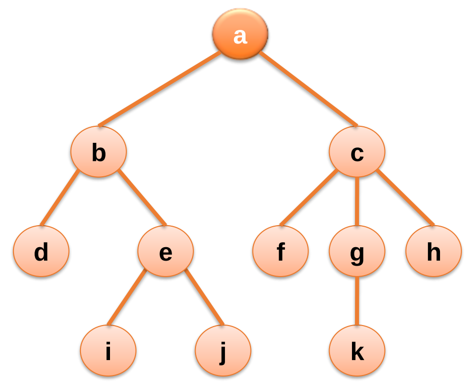


fit@hcmus

Tree Representation

	info	child
1	a	
2	b	
3	c	
4	d	
5	e	
6	f	
7	g	
8	h	
9	i	
10	j	
11	k	



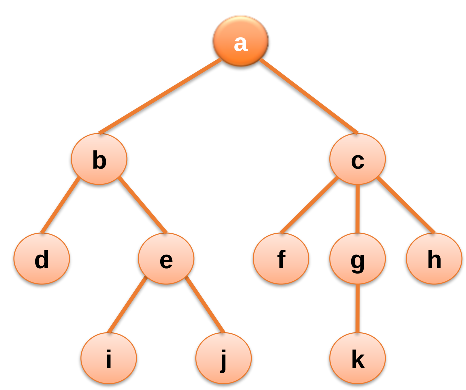


29

fit@hcmus

Tree Representation

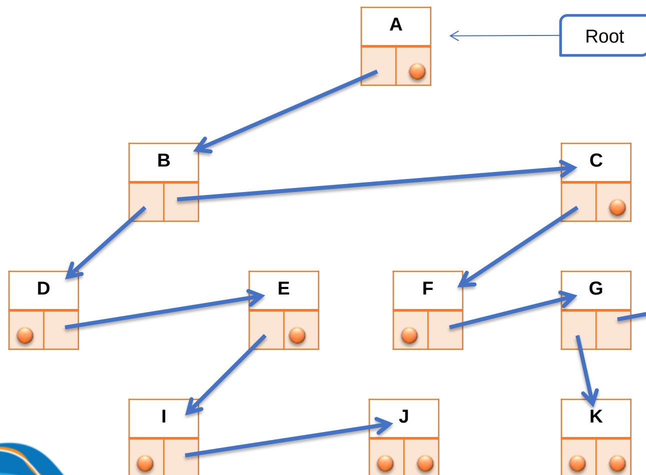
	Info	Eldest Child	Next Sibling
1	a	2	0
2	b	4	3
3	c	6	0
4	d	0	5
5	e	9	0
6	f	0	7
7	g	11	8
8	h	0	0
9	i	0	10
10	j	0	0
11	k	0	0



30

fit@hcmus

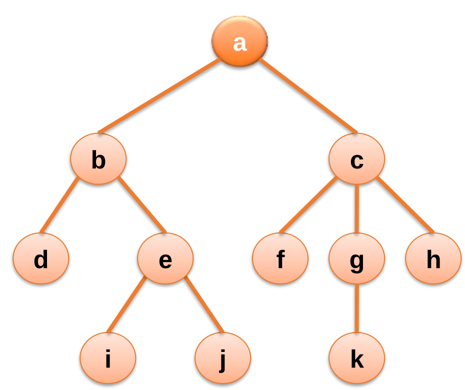
Tree Representation



fit@hcmus

Tree Representation

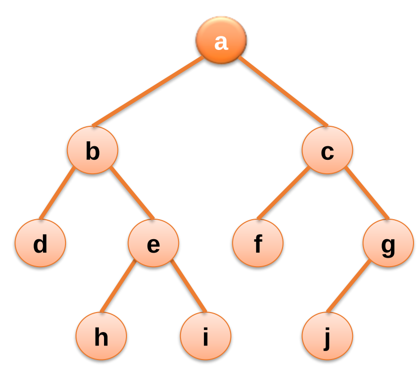
	Info	Parent
1	a	0
2	b	1
3	c	1
4	d	2
5	e	2
6	f	3
7	g	3
8	h	3
9	i	5
10	j	5
11	k	7



Binary Tree

Binary Tree

- Set T of nodes that is either empty or partitioned into disjoint subsets.
  - Single node *r*, the root
  - Two possibly empty sets that are binary trees, called *left* and *right* subtrees of *r*.
- Other definition: A rooted binary tree has a root node and every node has at most two children.



Types of Binary Tree

- Complete binary tree
- Full binary tree
- Perfect binary tree
- Heap

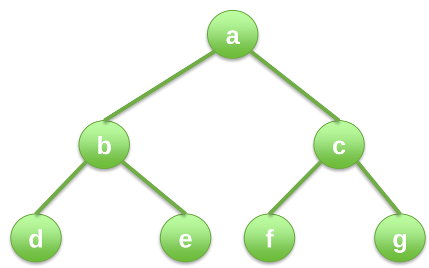
Perfect Binary Tree

- A **perfect binary tree** is a binary tree in which
  - all interior nodes have two children
  - and all leaves have the same depth or same level.
- In a perfect binary tree of height *h*, all nodes that are at a level less than *h* have two children each.

fit@hcmus

Perfect Binary Tree

- If T is empty, T is a perfect binary tree of height 0.
- If T is not empty and has height $h > 0$, T is a perfect binary tree if its root's subtrees are both perfect binary trees of height $h - 1$.



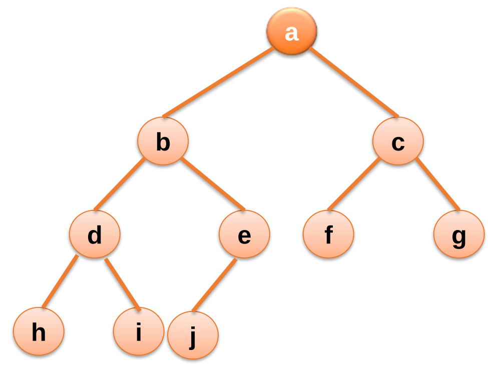
fit@hcmus

Complete Binary Tree

- A **complete binary tree** of height h is a binary tree that is **perfect** down to level $h - 1$, with level h filled in from left to right.
- In a complete binary tree every level, except possibly the last, is completely filled, and all nodes in the last level are as far left as possible.
- Other definition: A **complete binary tree** is a perfect binary tree whose rightmost leaves (perhaps all) have been removed.

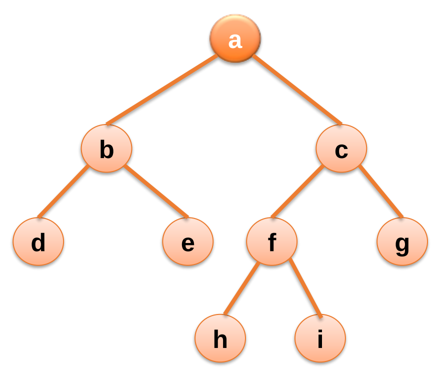
Complete Binary Tree

- A binary tree is complete if
  - All nodes at level *h* – 2 and above have two children each, and
  - When a node at level *h* – 1 has children, all nodes to its left at the same level have two children each, and
  - When a node at level *h* – 1 has one child, it is a left child



Full Binary Tree

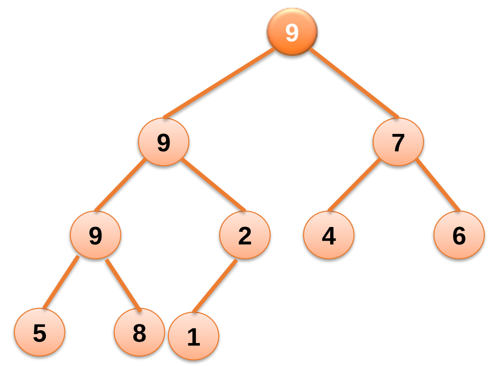
- A full binary tree (sometimes referred to as a **proper binary tree** or **a plane binary tree**) is a binary tree in which ***every node has either 0 or 2 children***.



- A full binary tree is either:
  - A single vertex.
  - A tree whose root node has two subtrees, both of which are full binary trees.

Heap

- A heap is a ***complete binary tree*** that either is empty or
  - Its root
    - (Max-heap): Contains a value greater than or equal to the value in each of its children, and
    - (Min-heap): Contains a value less than or equal to the value in each of its children, and
    - Has heaps as its subtrees



41

Number of Nodes

Given a binary tree *T* height of *h*.

- What is the maximum number of nodes?
- What is the minimum number of nodes?

42

Height of Tree

Given a binary tree *T* with *n* nodes.

- What is the maximum height of that tree?

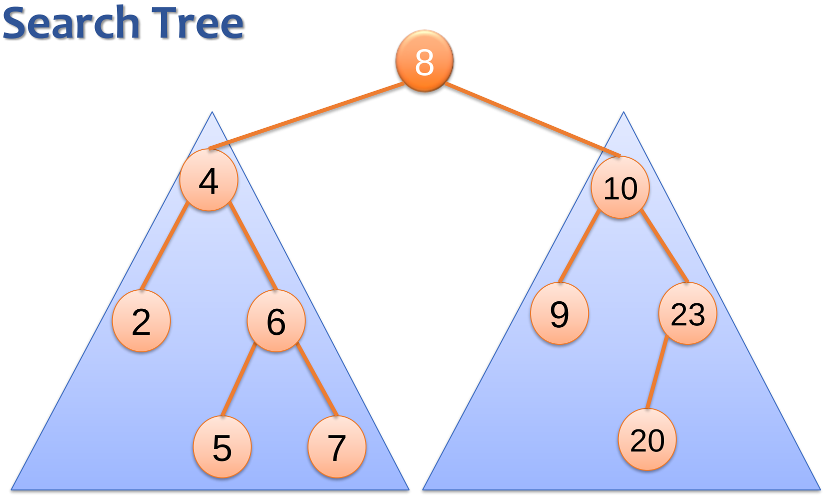
- What is the minimum height of that tree?

Binary Search Tree

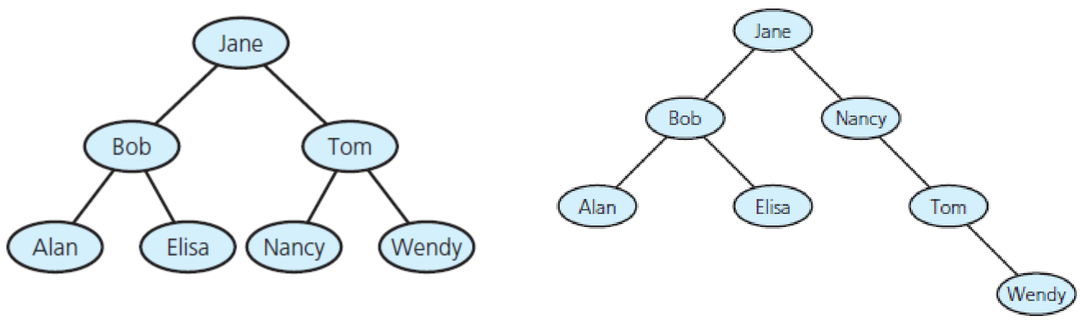
Binary Search Tree

- A binary search tree is a binary tree in which each node *n* has properties:
  - *n* 's value greater than all values in left subtree T_L
  - *n* 's value less than all values in right subtree T_R
  - Both T_R and T_L are binary search trees.

Binary Search Tree



Binary Search Tree



Operations

- Insert (a key)
- Search (a key)
- Remove (a key)
- Traverse
- Sort (based on key value)
- Rotate (Left rotation, Right rotation)

Insertion

```
Insert (root, Data)
{
      if (root is NULL) {
            Create a new_Node containing Data
            This new_Node becomes root of the tree
      }
      //Compare root's key with Key
      if root's Key is less than Data's Key
            Insert Key to the root's RIGHT subtree
      else if root's Key is greater than Data's Key
            Insert Key to the root's LEFT subtree
      else
            Do nothing //Explain why?
}
```


50

Insertion

Beginning with an empty binary search tree, what binary search tree is formed when you insert the following values in the order given?

15, 5, 12, 8, 23, 1, 17, 21

15, 20, 40, 25, 70, 90, 80, 55, 60, 65, 30, 75

9, 1, 4, 2, 3, 9, 5, 8, 6, 7, 4

51

Insertion

Beginning with an empty binary search tree, what binary search tree is formed when you insert the following values in the order given?

- W, T, N, J, E, B, A
- W, T, N, A, B, E, J
- A, B, W, J, N, T, E
- B, T, E, A, N, W, J

Search

```
Search (root, Data)
{
      if (root is NULL) {
            return NOT_FOUND;
      }
      //Compare root's key with Key
      if root's Key is less than Data's Key
            Search Data in the root's RIGHT subtree
      else if root's Key is greater than Data's Key
            Search Data in the root's LEFT subtree
      else
            return FOUND //Explain why?
}
```

Deletion

fit@hcmus

- When we delete a node, three possibilities arise.
- Node to be deleted:
  - **is leaf:**
    - `Simply remove from the tree.`
  - `has` **only one child:**
    - `Copy the child to the node and delete the child`
  - `has` **two children:**
    - `Find in-order successor (predecessor)` ***S_Node*** `of the node.`
    - `Copy contents of` ***S_Node*** `to the node and delete the` ***S_Node***`.`

Traversals

fit@hcmus

- Pre-order:

  Node - Left - Right

- In-order:

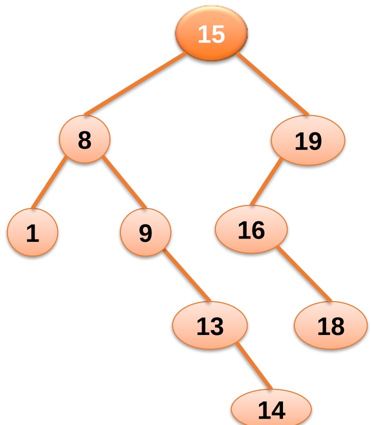
  Left - Node - Right

- Post-order:

  Left - Right - Node

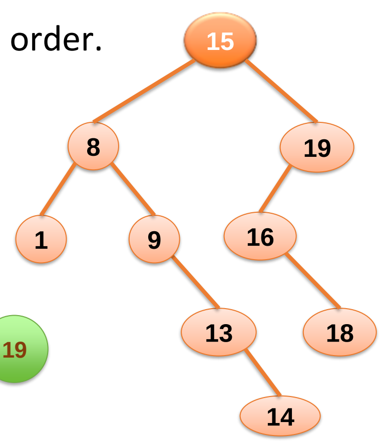
Traversals

What are the pre-order, in-order and post-order traversals of this binary search tree?

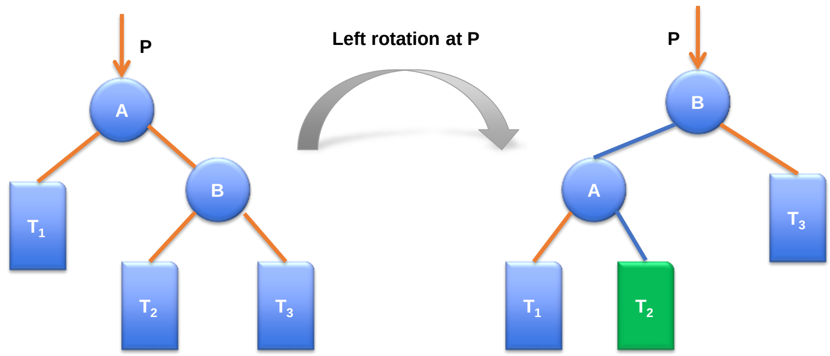


Sort

In-order traversal gives the key values in sorted order.

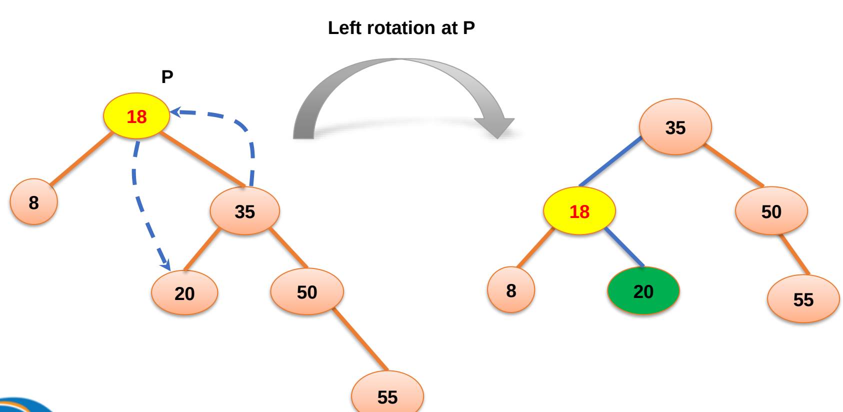


Left Rotation



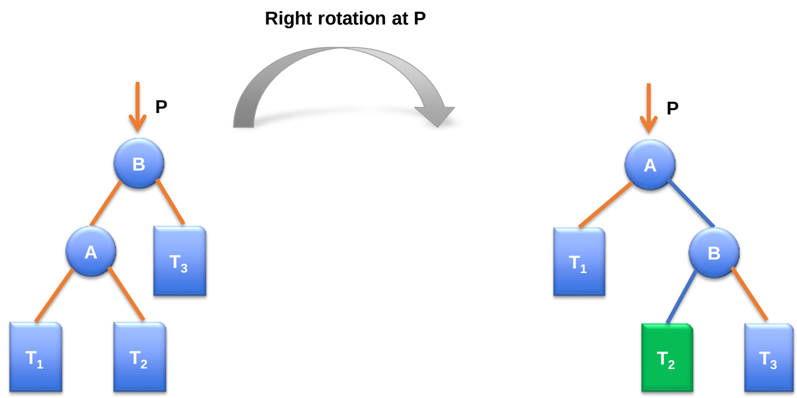
58

Left Rotation

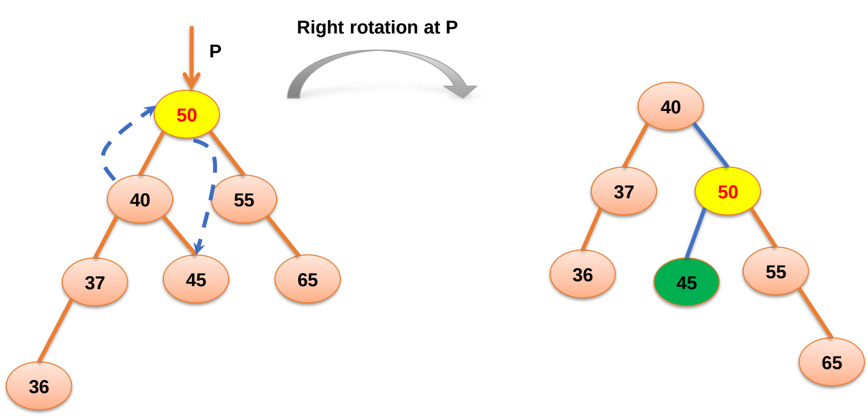


59

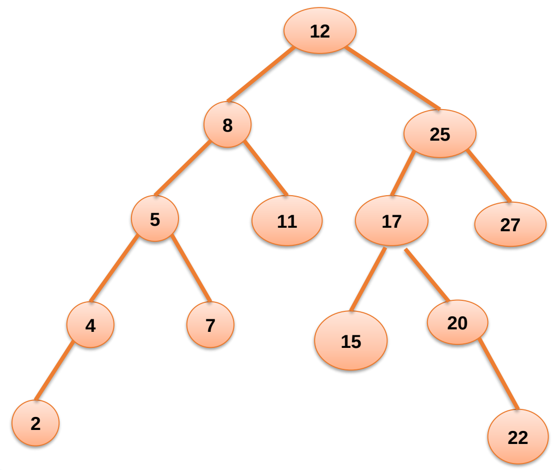
Right Rotation



Right Rotation



Binary Search Tree



Efficiency of Binary Search Tree Operations

Operation	Average case	Worst case
Retrieval	O(log *n*)	O(*n*)
Insertion	O(log *n*)	O(*n*)
Removal	O(log *n*)	O(*n*)
Traversal	O(*n*)	O(*n*)

Very Bad Binary Search Tree

Beginning with an empty binary search tree, what binary search tree is formed when inserting the following values in the order given?

2, 4, 6, 8, 10, 12, 14, 18, 20

AVL Tree

fit@hcmus

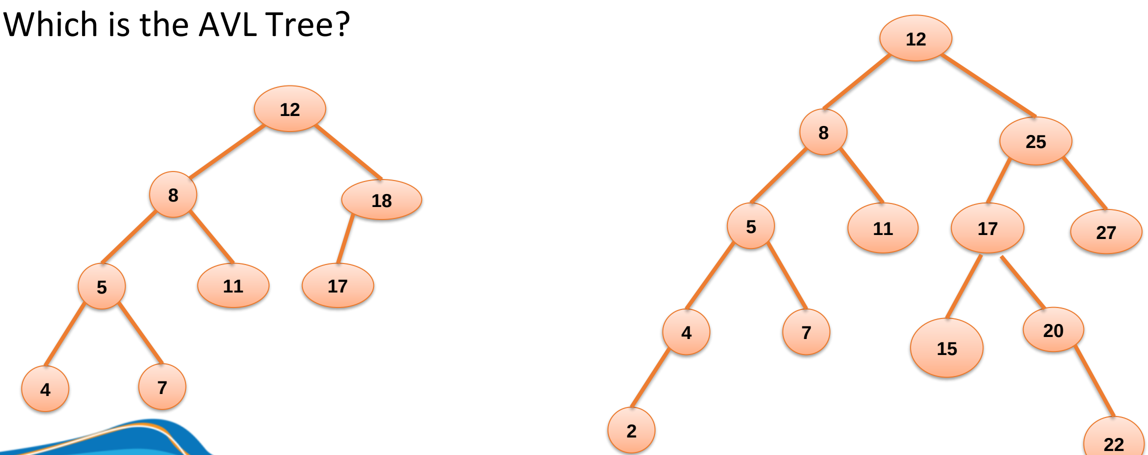
AVL Tree

- Named for its inventors, (Georgii) **A**delson-**V**elsky and (Evgenii) **L**andis
- Invented in 1962 (paper “*An algorithm for organization of information*”).
- AVL Tree is a **self-balancing** binary search tree where
  - for ALL nodes, the difference between height of the left subtrees and the right subtrees **cannot be more than one**. (*height invariant*, or *balance invariant*).

fit@hcmus

AVL Tree

Which is the AVL Tree?



AVL Tree

- A balanced binary search tree
  - Maintains height close to the minimum
  - After insertion or deletion, check the tree is still AVL tree – determine whether any node in tree has left and right subtrees whose heights differ by more than 1
- Can search AVL tree almost as efficiently as minimum-height binary search tree.

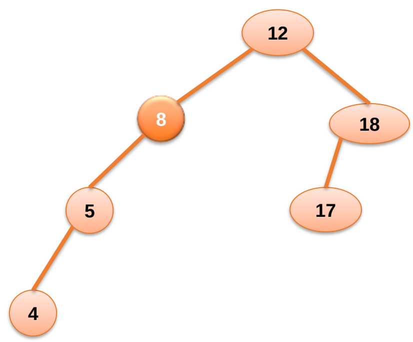
Cases of Height Invariant Violation

- Left-Left case
- Left-Right case
- Right-Right case
- Right-Left case

fit@hcmus

Cases of Height Invariant Violation

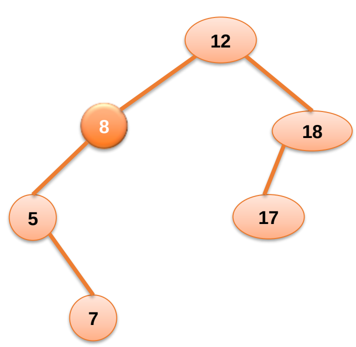
- Left-Left case



fit@hcmus

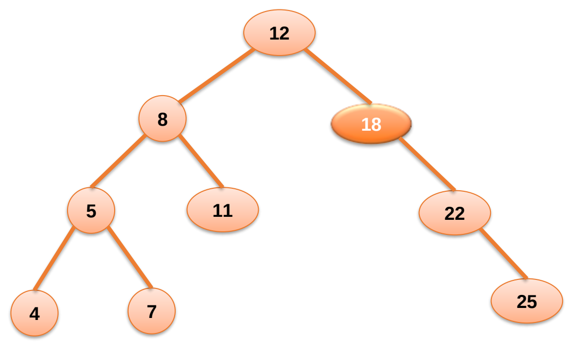
Cases of Height Invariant Violation

- Left-Right case



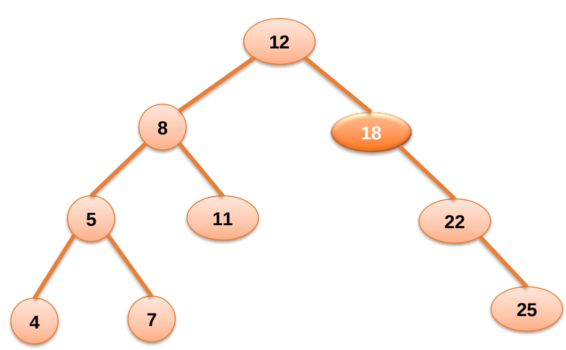
Cases of Height Invariant Violation

- Right-Right case



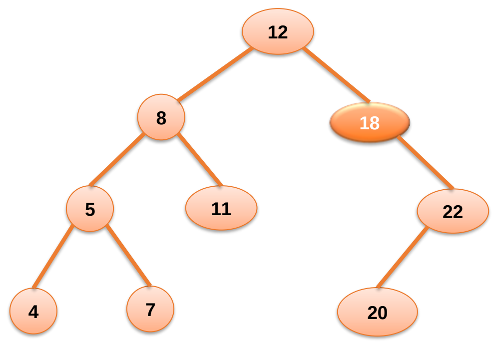
Cases of Height Invariant Violation

- Right-Right case



Cases of Height Invariant Violation

- Right-Left case



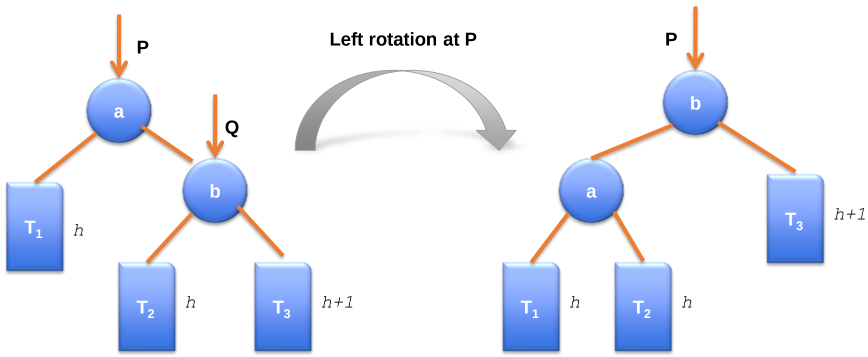
Violation Resolving

- Right-Right case:
  - Left rotation at un-balanced node.
- Right-Left case:
  - Right rotation at un-balanced node's right child.
  - Left rotation at un-balanced node.

fit@hcmus

Violation Resolving

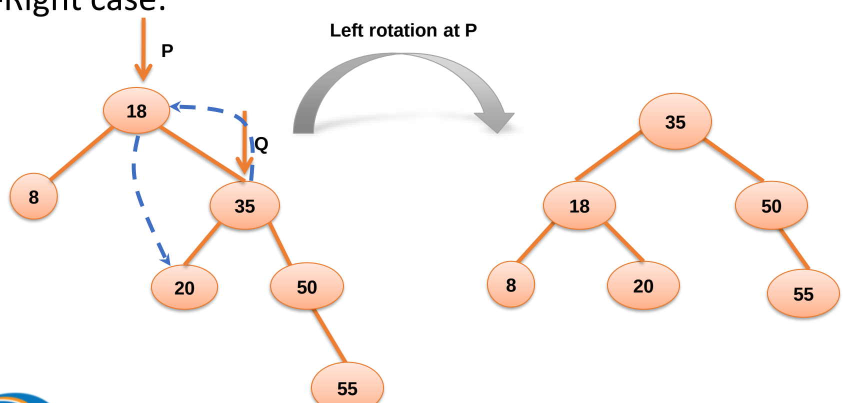
- Right-Right case:



fit@hcmus

Violation Resolving

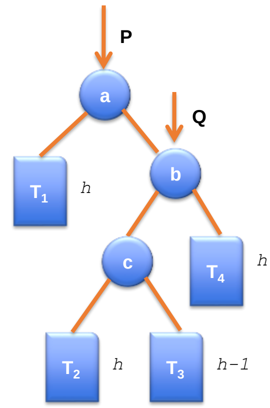
- Right-Right case:



fit@hcmus

Violation Resolving

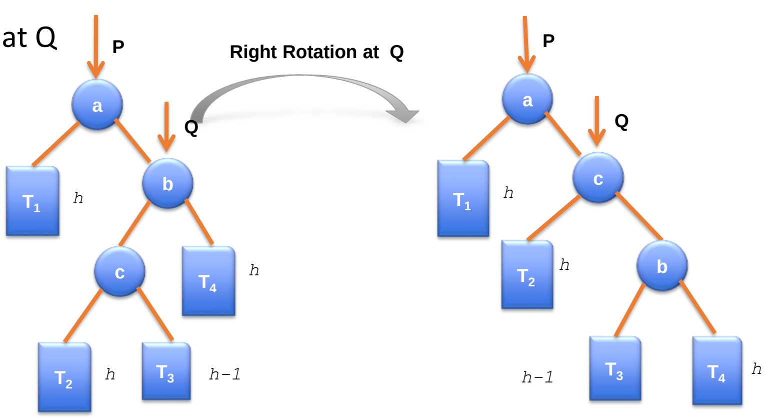
- Right-Left case:
  - Right rotation at Q
  - Left rotation at P



fit@hcmus

Violation Resolving

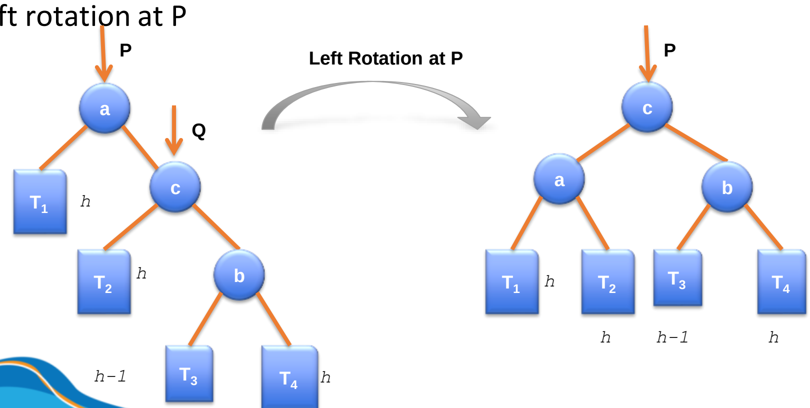
- Right-Left case:
  - Right rotation at Q



fit@hcmus

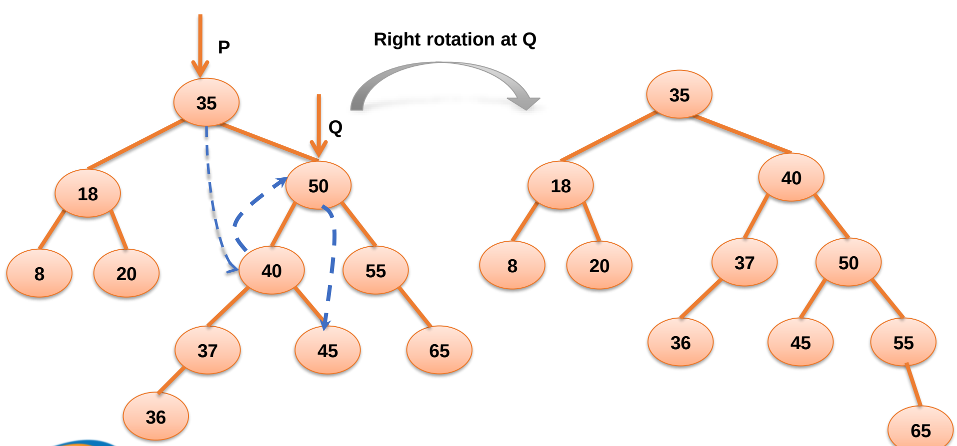
Violation Resolving

- Right-Left case:
  - Left rotation at P



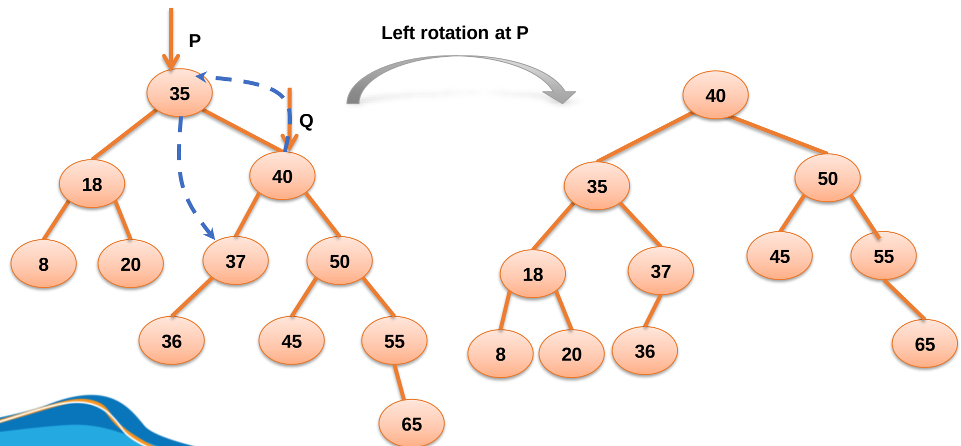
fit@hcmus

Violation Resolving



fit@hcmus

Violation Resolving



fit@hcmus

Violation Resolving

- Left-Left case:

  Single rotation

  - Right rotation at un-balanced node.

- Left-Right case:

  Double rotation

  - Left rotation at un-balanced node's left child.
  - Right rotation at un-balanced node.

Violation Resolving

Source: Wikipedia

84

Insertion

Beginning with an empty AVL tree, perform step-by-step the insertion of the following values in the order given

15, 5, 12, 8, 23, 1, 17, 21

9, 1, 4, 2, 3, 9, 5, 8, 6, 7, 4

85

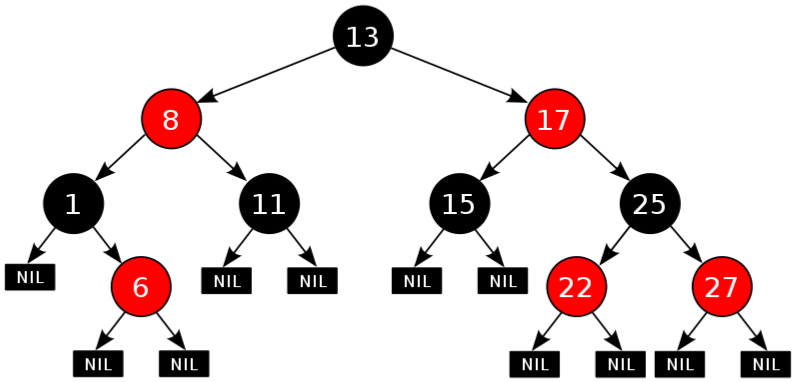
Red-Black Tree

fit@hcmus

Red-Black Tree

- Invented in 1972 by Rudolf Bayer.
- Red-Black tree is a binary search tree with the following rules:
  - Every node has a color either **red** or **black**.
  - The root of the tree is always **black**.
  - There are no two adjacent **red** nodes (A **red** node cannot have a **red** parent or **red** child).
  - Every path from a node (including root) to any of its descendants NULL nodes has the same number of **black** nodes.
  - All leaf nodes are **black** nodes.

Red-Black Tree



AA Tree

fit@hcmus

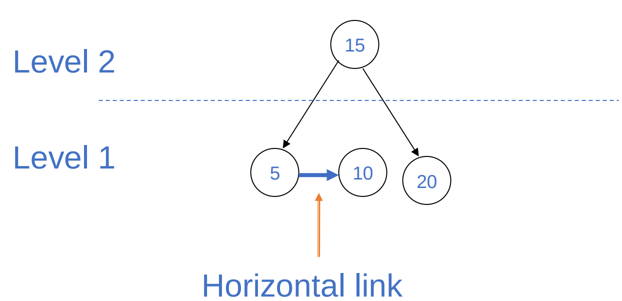
AA Tree

- Invented by **A**rne **A**nderson in a work published in 1993 (Balanced Search Tree Made Simple).

- Two concepts:
  - Level:
    - Number of LEFT links from that nodes to a NULL node.
  - Horizontal link:
    - The link between parent and its child node having the same level.

fit@hcmus

AA Tree



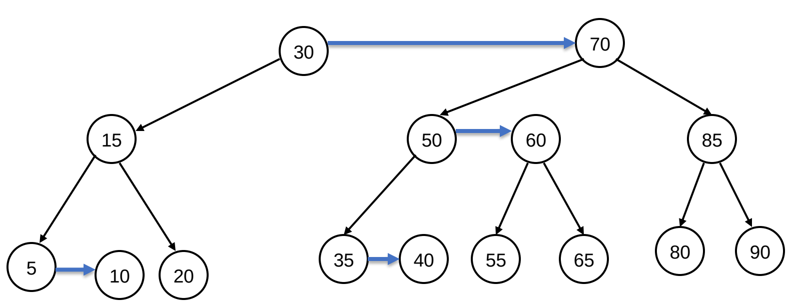
AA Tree

- AA tree is a binary search tree with the following rules:
  - The level of every leaf node is one.
  - The level of every left child is exactly one less than that of its parent.
  - The level of every right child is equal to or one less than that of its parent.

    Horizontal link must be a RIGHT link.
  - The level of every right grandchild is strictly less than that of its grandparent.

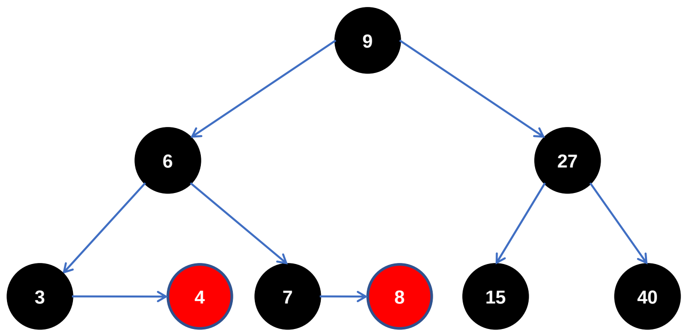
    There is no two consecutive horizontal links.
  - Every node of level greater than one has two children.

AA Tree



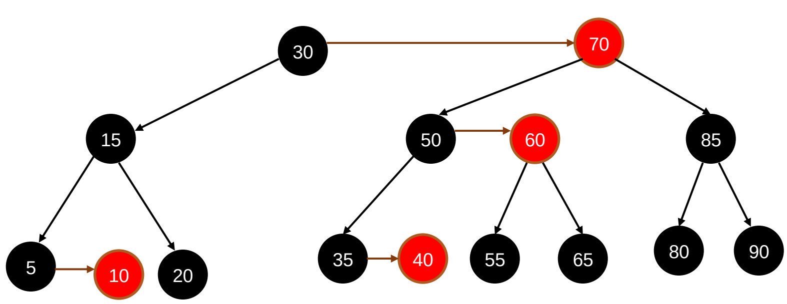
fit@hcmus

AA Tree and Red-Black Tree



fit@hcmus

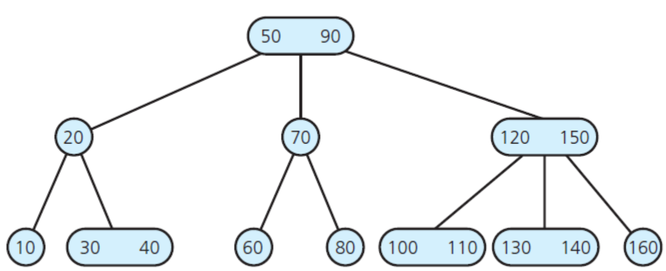
AA Tree and Red-Black Tree



2-3 Tree and 2-3-4 Tree

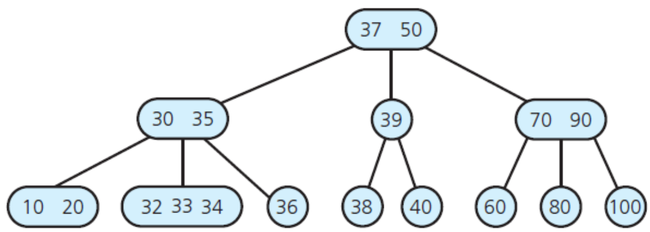
fit@hcmus

2-3 Tree



fit@hcmus

2-3-4 Tree



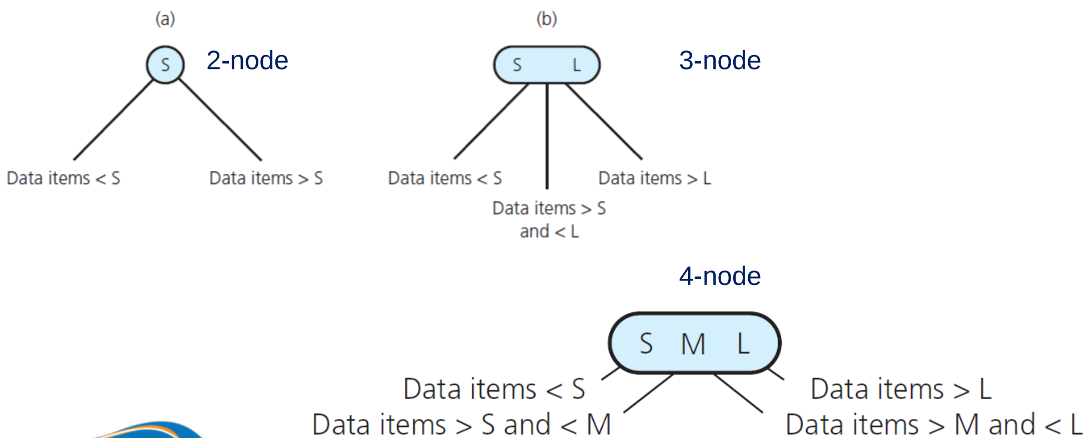
fit@hcmus

2-node, 3-node, 4-node

- A `2-node` (has two children) must contain **single** data item greater than left child's item(s) and less than right child's item(s).
- A `3-node` (has three children) must contain **two** data items, `S` and `L`, such that
  - S is greater than left child's item(s) and less than middle child's item(s);
  - L is greater than middle child's item(s) and less than right child's item(s).
- A `4-node` (has our children) must contain **three** data items `S`, `M`, and `L` that satisfy:
  - S is greater than left child's item(s) and less than middle-left child's item(s)
  - M is greater than middle-left child's item(s) and less than middle-right child's item(s);
  - L is greater than middle-right child's item(s) and less than right child's item(s).

fit@hcmus

2-node, 3-node, 4-node



fit@hcmus

2-3 Tree, 2-3-4 Tree

- A 2-3 tree is not a binary tree. Neither is a 2-3-4 tree.
- A 2-3 tree, a 2-3-4 tree are never taller than a minimum-height binary tree.

fit@hcmus

2-3 Tree

- Invented by John Hopcroft in 1970.

- 2-3 tree is a tree in which
  - Every internal node is either a 2-node or a 3-node.
  - Leaves have no children and may contain either one or two data items.

fit@hcmus

2-3-4 Tree

- 2-3-4 tree is a tree in which
  - Every internal node is a 2-node, a 3-node or a 4-node.
  - Leaves have no children and may contain either one, two or three data items.

Insertion (2-3 Tree)

fit@hcmus

Inserting Data into a 2-3 Tree

```
// Inserts a new item into a 2-3 tree whose items are distinct and differ from the
// new item.
insertItem(23Tree: TwoThreeTree, newItem: ItemType)

   Locate the leaf, leafNode, in which newItem belongs
   Add newItem to leafNode

   if (leafNode has three items)
      split(leafNode)

// Splits node n, which contains two items. Note: If n is
// not a leaf, it has four children.
split(n: TwoThreeNode)

   if (n is the root)
      Create a new node p
   else
      Let p be the parent of n

   Replace node n with two nodes, n1 and n2, so that p is their parent
   Give n1 the item in n with the smallest value
```

fit@hcmus

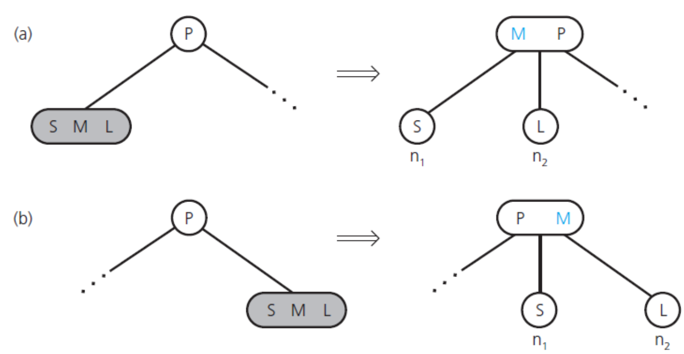
Inserting Data into a 2-3 Tree

```
split(n: TwoThreeNode)

  if (n is the root)
     Create a new node p
  else
     Let p be the parent of n

  Replace node n with two nodes, n1 and n2, so that p is their paren
  Give n1 the item in n with the smallest value
  Give n2 the item in n with the largest value

  if (n is not a leaf)
  {
     n1 becomes the parent of n's two leftmost children
     n2 becomes the parent of n's two rightmost children
  }
  Move the item in n that has the middle value up to p
  if (p now has three items)
     split(p)
```


fit@hcmus

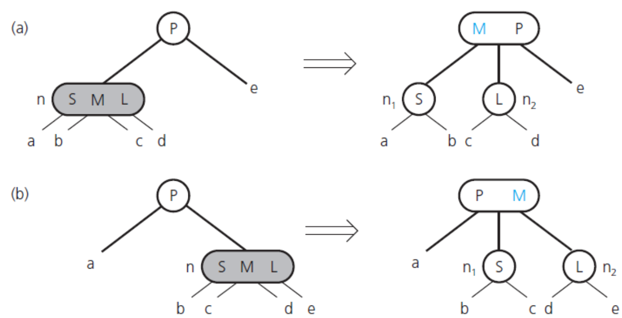
Inserting Data into a 2-3 Tree

- Splitting a leaf in a 2-3 tree when the leaf is a
  - (a) left child; (b) right child



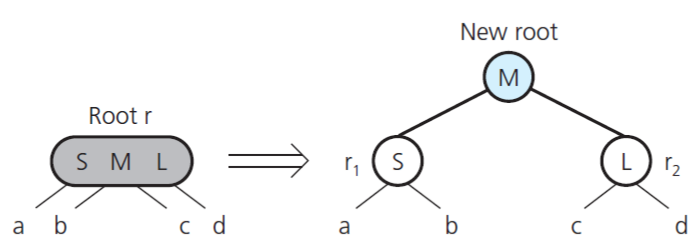
Inserting Data into a 2-3 Tree

- Splitting an internal node in a 2-3 tree when the node is a (a) left child; (b) right child



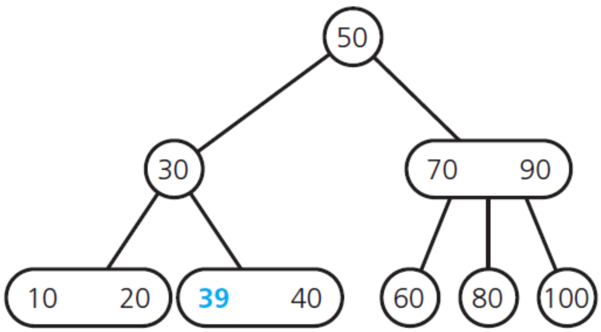
Inserting Data into a 2-3 Tree

- Splitting the root of a 2-3 tree



Examples

- After inserting **39** into the tree



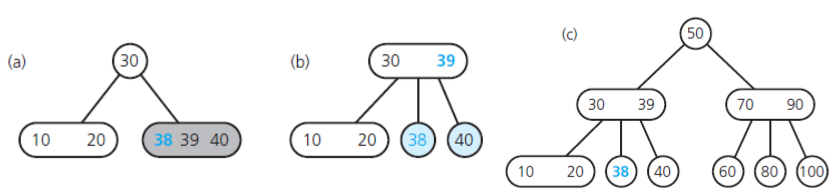
Examples

The steps for inserting **38** into the tree

(a) The located node has no room;

(b) the node splits;

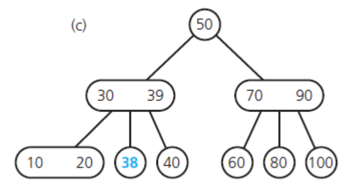
(c) the resulting tree



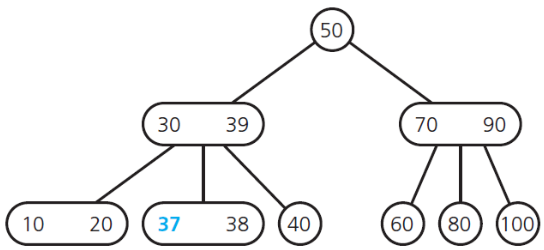
Next: inserting **37** into the tree

Examples

After inserting **37** into the tree



Original tree



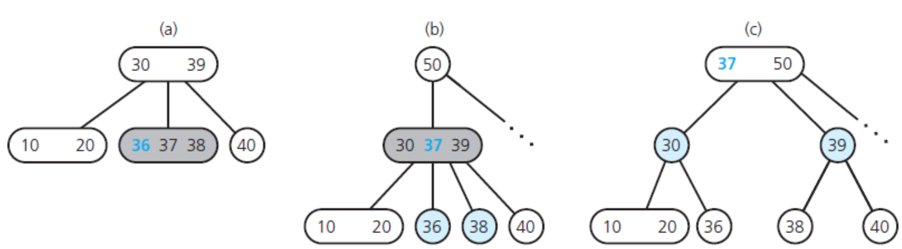
Resulting tree

Next: inserting **36** into the tree

121

Examples

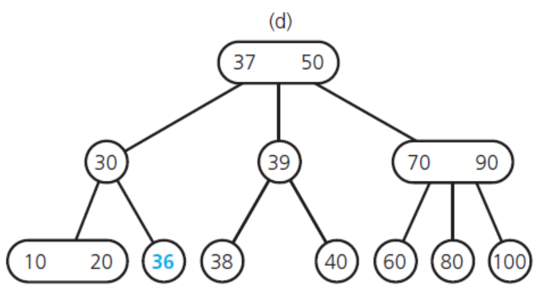
The steps for inserting 36 into the tree



122

Examples

the resulting tree



Insertion (2-3-4 Tree)

Inserting Data into a 2-3-4 Tree

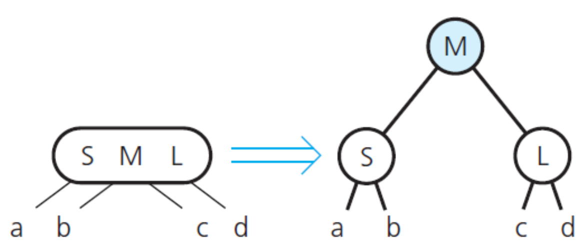
- Insertion algorithm splits a node by moving one of its items up to its parent node

- **Splits** 4-nodes ***as soon as*** it encounters them on the way down the tree from the root to a leaf

125

Inserting Data into a 2-3-4 Tree

- Splitting a 4-node root during insertion into a 2-3-4 tree

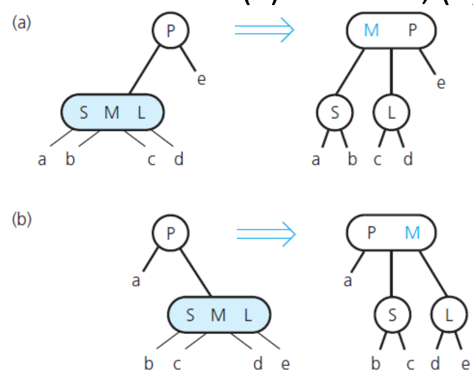


126

fit@hcmus

Inserting Data into a 2-3-4 Tree

- Splitting a 4-node whose parent is a 2-node during insertion into a 2-3-4 tree, when the 4-node is a (a) left child; (b) right child

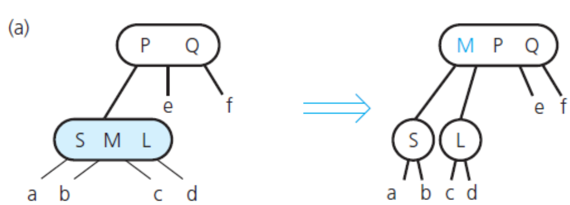


127

fit@hcmus

Inserting Data into a 2-3-4 Tree

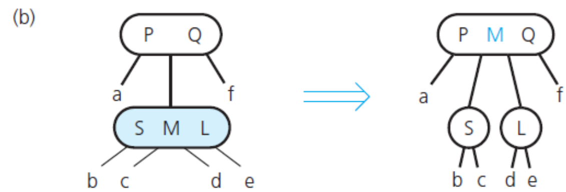
- Splitting a 4-node whose parent is a 3-node during insertion into a 2-3-4 tree, when the 4-node is a (a) left child



128

Inserting Data into a 2-3-4 Tree

- Splitting a 4-node whose parent is a 3-node during insertion into a 2-3-4 tree, when the 4-node is a (b) middle child

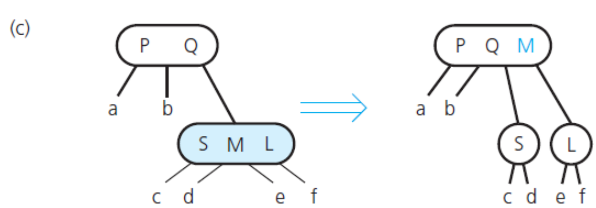


129

fit@hcmus

Inserting Data into a 2-3-4 Tree

- Splitting a 4-node whose parent is a 3-node during insertion into a 2-3-4 tree, when the 4-node is a (c) right child



130

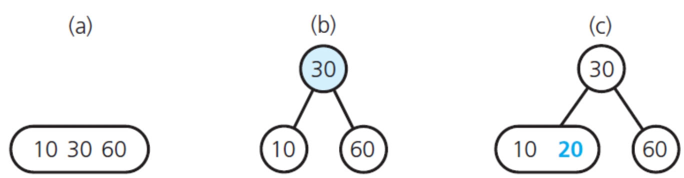
Examples

- Inserting **20** into a 2-3-4 tree

  (a) the original tree;

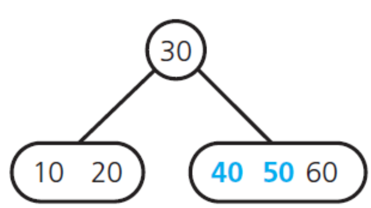
  (b) after splitting the node;

  (c) after inserting 20



Examples

- After inserting **50** and **40** into the tree

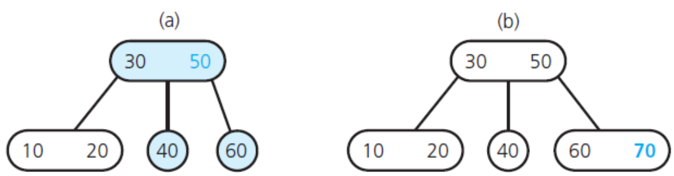


Next: inserting **70** into the tree

fit@hcmus

Examples

- The steps for inserting **70** into the tree
  (a) after splitting the 4-node;
  (b) after inserting 70



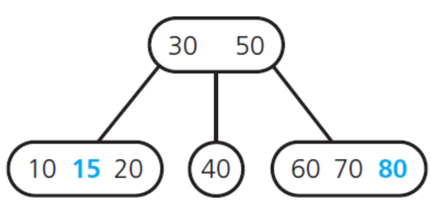
Next: inserting **80,** and **15** into the tree

133

fit@hcmus

Examples

- After inserting **80** and **15** into the tree

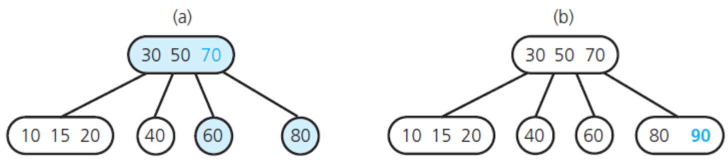


Next: inserting **90** into the tree

134

Examples

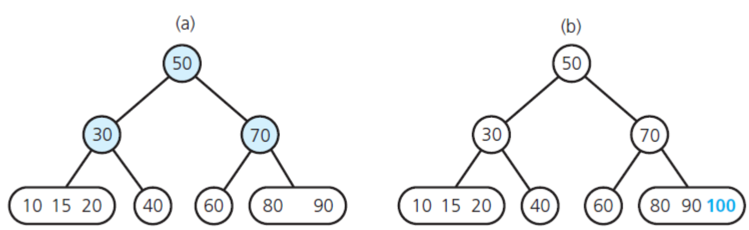
- The steps for inserting **90** into the tree



Next: inserting **100** into the tree

Examples

- The steps for inserting **100** into the tree



Deletion

fit@hcmus

Removing Data from a 2-3 Tree

```
// Removes the given data item from a 2-3 tree. Returns true if successful
// or false if no such item exists.
removeItem(23Tree: TwoThreeTree, dataItem: ItemType): boolean

   Attempt to locate dataItem
   if (dataItem is found)
   {
      if (dataItem is not in a leaf)
         Swap dataItem with its inorder successor, which will be in a leaf leafNode

      // The removal always begins at a leaf
      Remove dataItem from leaf leafNode
      if (leafNode now has no items)
         fixTree(leafNode)
      return true
   }
   else
      return false
```

Removing Data from a 2-3 Tree

fit@hcmus

```
else
    return false

// Completes the removal when node n is empty by either deleting the root,
// redistributing values, or merging nodes. Note: If n is internal, it has one child
fixTree(n: TwoThreeNode)

  if (n is the root)
      Delete the root
  else
  {
     Let p be the parent of n
     if (some sibling of n has two items)
     {
         Distribute items appropriately among n, the sibling, and p
```


Removing Data from a 2-3 Tree

fit@hcmus

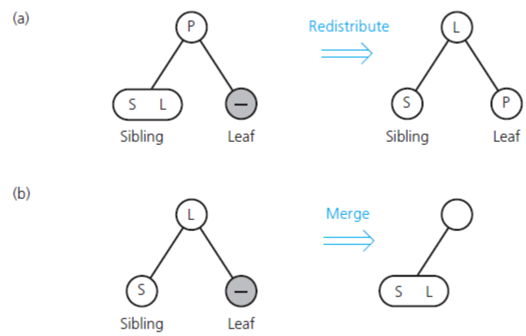
```
        if (n is internal)
            Move the appropriate child from sibling to n
     }
     else // Merge the node
     {
        Choose an adjacent sibling s of n
        Bring the appropriate item down from p into s
        if (n is internal)
            Move n's child to s
        Remove node n
        if (p is now empty)
            fixTree(p)
     }
  }
```

fit@hcmus

Removing Data from a 2-3 Tree

(a) Redistributing values;

(b) merging a leaf;

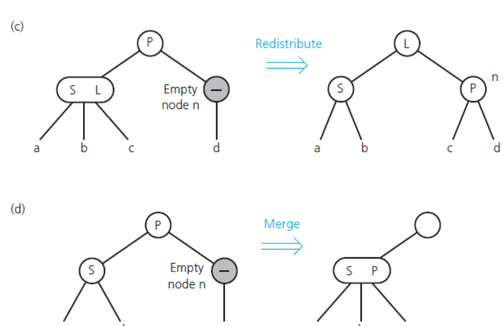


fit@hcmus

Removing Data from a 2-3 Tree

(c) redistributing values and children

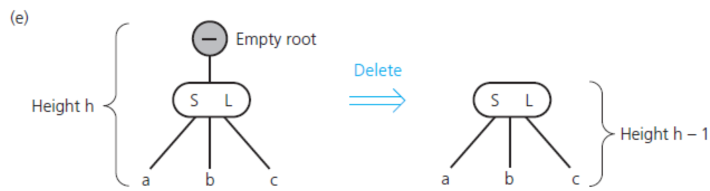
(d) merging internal nodes



fit@hcmus

Removing Data from a 2-3 Tree

(e) deleting the root

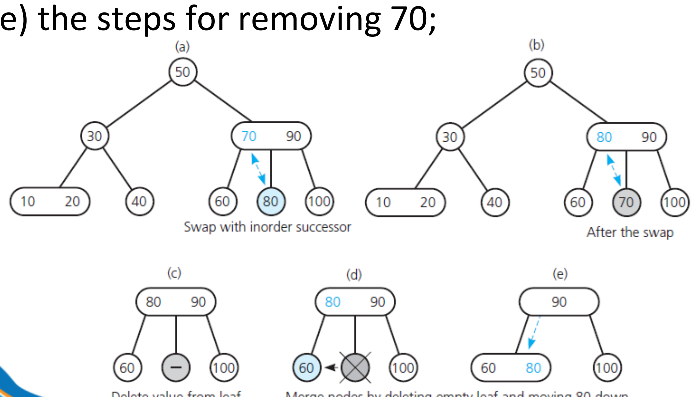


fit@hcmus

Examples

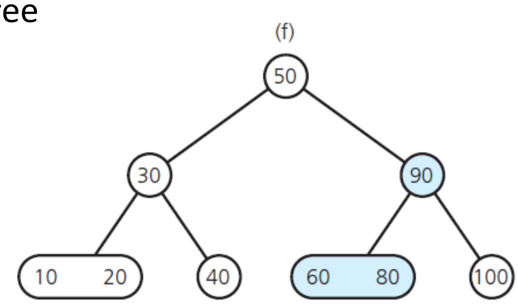
(a) A 2-3 tree;

(b), (c), (d), (e) the steps for removing 70;



Examples

- the resulting tree

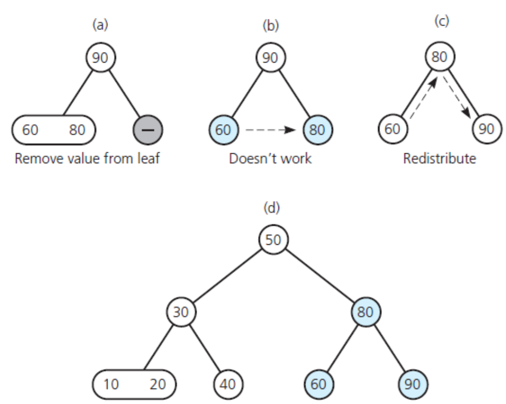


146

Examples

(a), (b), (c) The steps for removing 100 from the tree;

(d) the resulting tree

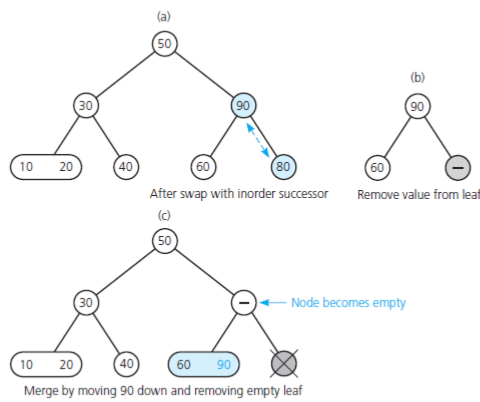


147

fit@hcmus

Examples

The steps for removing 80 from the tree

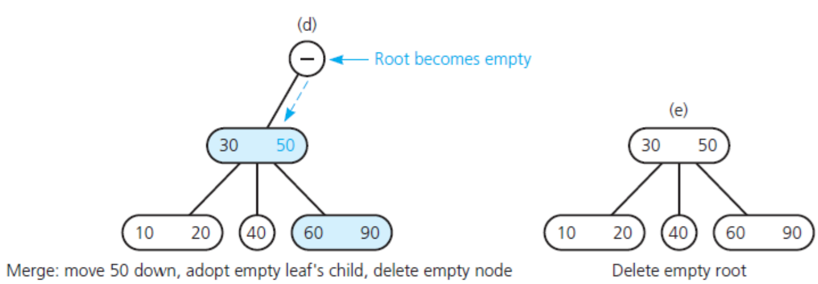


148

fit@hcmus

Examples

The steps for removing 80 from the tree



149

Removing Data from a 2-3-4 Tree

- Removal algorithm has the same beginning as removal algorithm for a 2-3 tree
- Locate the node `n` that contains the item `I` you want to remove.
- Find `I` 's in-order successor and swap it with `I` so that the removal will always be at a leaf.
- If leaf is either a 3-node or a 4-node, remove `I` .

B-Tree

B-Tree

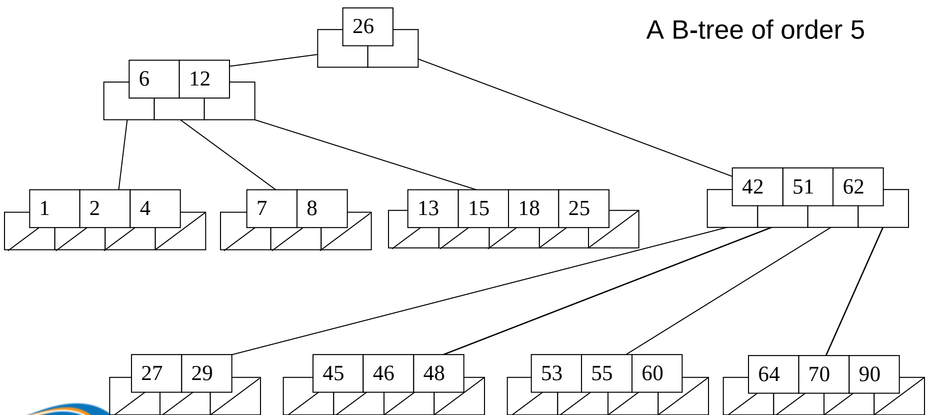
- Original B-Tree proposed by R. Bayer and E. McCreigh in 1972.
- A B-Tree is a specialized multi-way tree designed especially for use on external disk.
- Improved versions of B-Trees were later proposed in 1982 by Huddleston and Mehlhorn, and by Maier and Salveter.
- B-tree variants are used mostly today as index structures in database applications.

Definition

- *(Knuth's definition)* A B-tree of order *m* is a tree which satisfies the following properties:
  - Every node has at most *m* children.
  - Every non-leaf node (except root) has at least $\lceil m/2 \rceil$ child nodes.
  - The root has **at least two children** if it is not a leaf node.
  - A non-leaf node with *k* children contains ***k* – 1** keys.
  - All leaves appear in the same level and carry no information.
- The number *m* should be (always) odd.

Example

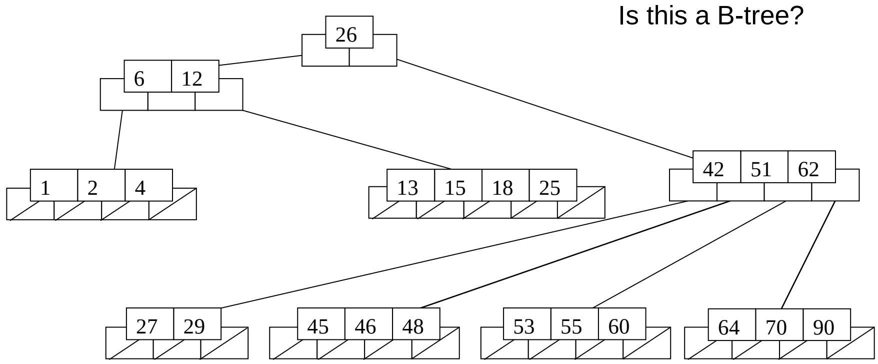
A B-tree of order 5



163

Example

Is this a B-tree?



164

Height of B-Tree

- The maximum number of keys in a B-tree of order *m* and height *h*:

root	$m-1$
level 2	$m(m-1)$
level 3	$m^2(m-1)$
. . .	
level h	$m^{h-1}(m-1)$

- So, the total number of keys is

$$(1 + m + m^2 + m^3 + \ldots + m^{h-1})(m-1) =$$
$$[(m^h - 1)/(m-1)]\,(m-1) = \mathbf{m^h - 1}$$

Operations

- Insert (a key)
- Remove (a key)

Insertion

fit@hcmus

Insertion

- B-tree insertion is a generalization of 2-3 tree insertion.
- Insert K into B-tree of order m.
  - We find the insertion point (in a leaf) by doing a search.
  - If there is room then insert K.
  - Else, promote the middle key to the parent, split the node into nodes around the middle key.
- If the splitting backs up to the root, then
  - Make a new root containing the middle key.
- Note
  - The tree grows from the leaves, balance is always maintained.

Example

- Perform step-by-step the insertion of the following values to the initially empty B-tree order 5

1, 12, 8, 2, 25, 5, 14, 28, 17

169

1 12 8 2 25 5 14 28 17

Example

1	2	8	12

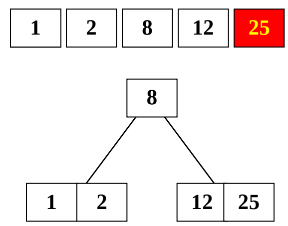
Insert **1** Insert **12** Insert **8** Insert **2**

170

1 12 8 2 **25** 5 14 28 17

fit@hcmus

Example



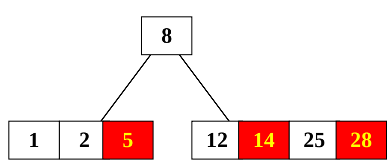
Insert **25**

171

1 12 8 2 25 **5** **14** **28** 17

fit@hcmus

Example



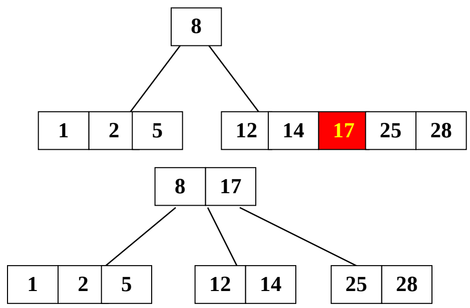
Insert **5, 14, 28**

172

1 12 8 2 25 5 14 28 **17**

fit@hcmus

Example



Insert **17**

Deletion

Deletion

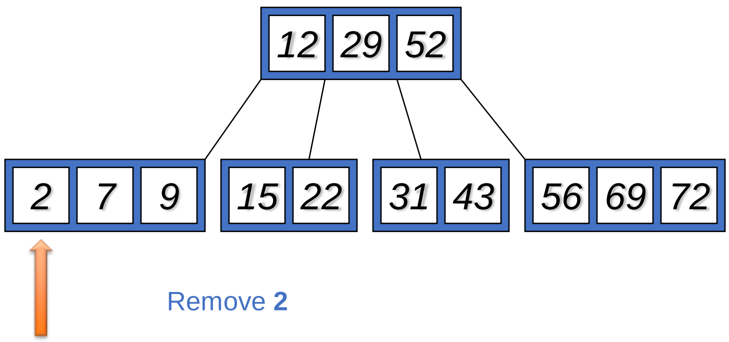
- If the key to be deleted is not in a leaf, swap it with its successor (or predecessor). Then delete the key from the leaf.
- If leaf contains more than the minimum number of keys, then one can be deleted with no further action.

Deletion

- If the node contains the minimum number of keys, consider the two immediate siblings of the parent node:
  - If one of these siblings has more than the minimum number of keys, then **redistribute** one key from this sibling to the parent node, and one key from the parent to the deficient node.
  - If both immediate siblings have exactly the minimum number of keys, then **merge** the deficient node with one of the immediate sibling node and one key from the parent node.
- If this leaves the parent node with too few keys, then the process is propagated upward.

Example

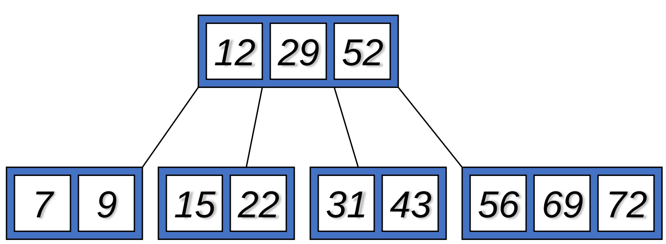
Given a B-tree order 5:



Remove **2**

Example

Given a B-tree order 5:

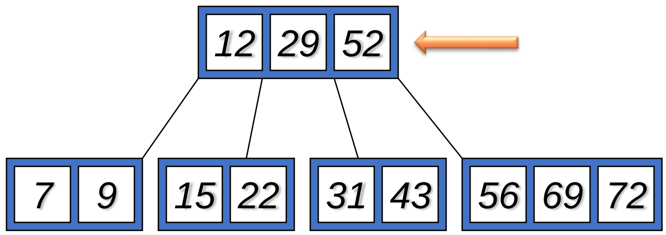


After removing **2**

fit@hcmus

Example

Given a B-tree order 5:

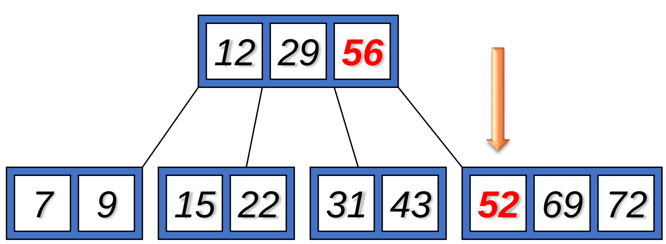


Remove **52**

fit@hcmus

Example

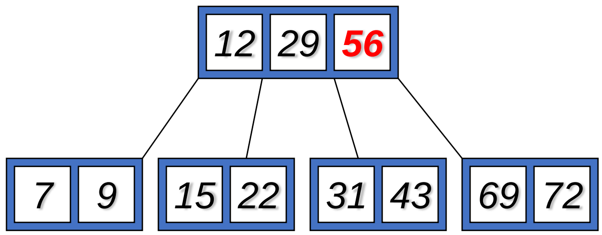
Given a B-tree order 5:



Remove **52** (replaced by **56**)

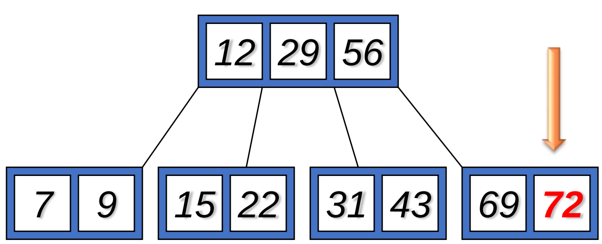
Example

Given a B-tree order 5:



Example

Given a B-tree order 5:

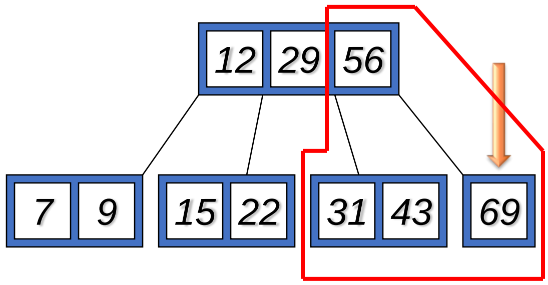


Remove **72**

fit@hcmus

Example

Given a B-tree order 5:



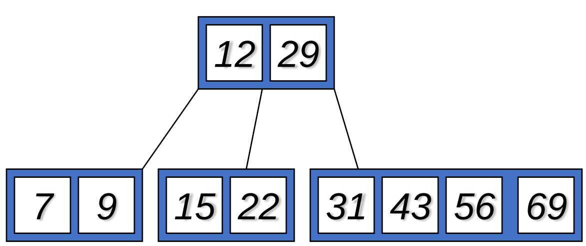
Not enough key. **Merge** nodes.

183

fit@hcmus

Example

Given a B-tree order 5:

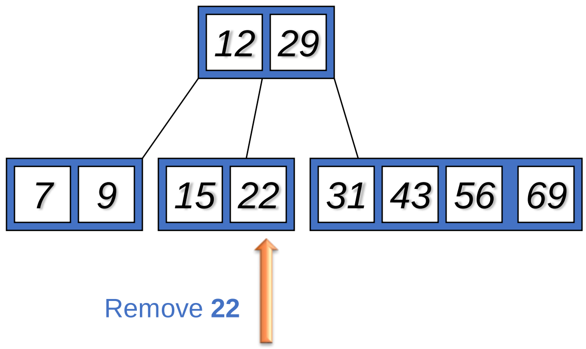


184

fit@hcmus

Example

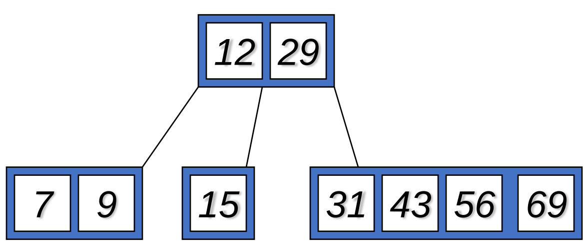
Given a B-tree order 5:



fit@hcmus

Example

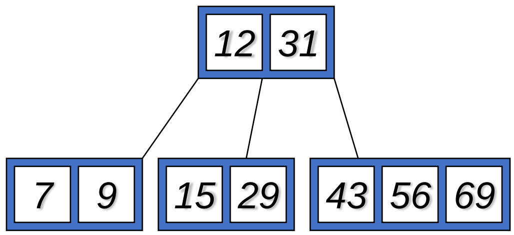
Given a B-tree order 5:



(**Redistribute**) Borrow key from neighboring nodes

Example

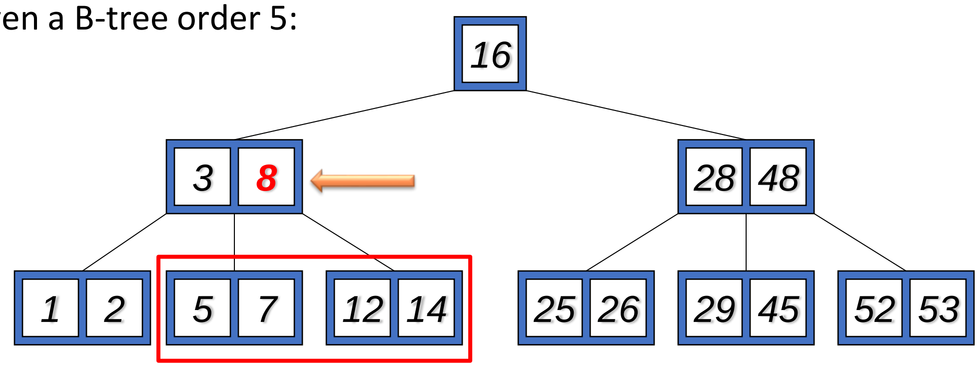
Given a B-tree order 5:



187

Example

Given a B-tree order 5:



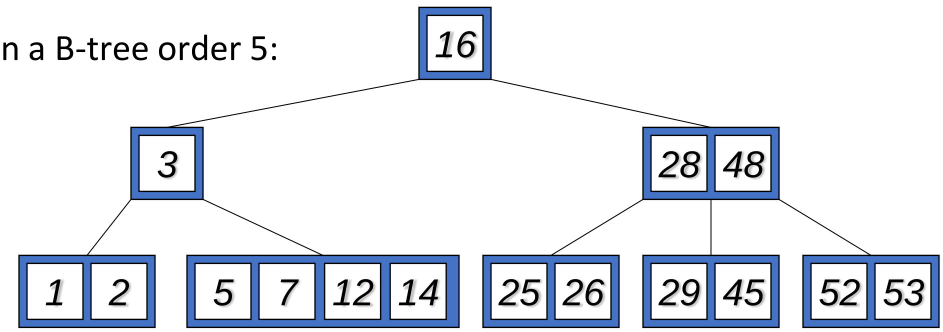
Remove 8 → Merge 2 child nodes of 8

188

fit@hcmus

Example

Given a B-tree order 5:

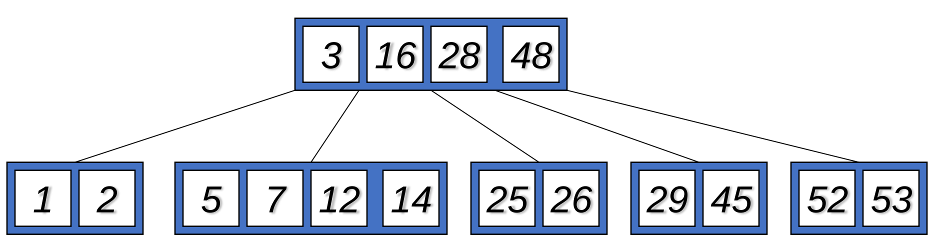


Not enough keys at node 3.

fit@hcmus

Example

Given a B-tree order 5



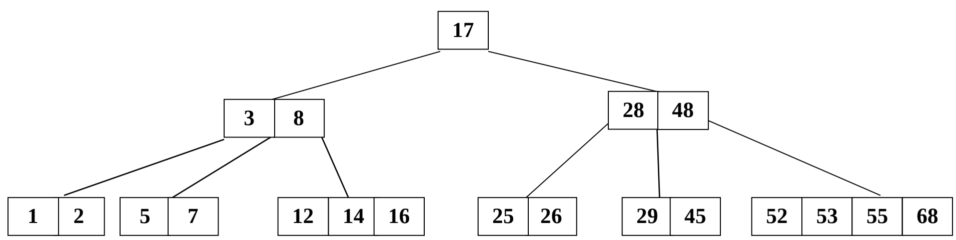
A new root node

Decrease height of the tree

Quiz

Given the following B-tree order 5

- Remove 28, then remove 48



B-Trees & Efficiency

- Used in Mac, NTFS, OS2 for file structure.
- Allow insertion and deletion into a tree structure, based on $\log_m n$ property, where *m* is the order of the tree.
- The idea is that you leave some key spaces open. An insertion of a new key is done using available space (most cases).
  - Less dynamic than our typical Binary Tree
  - Ideal for disk-based operations.

B-Trees & Efficiency

fit@hcmus

- In practical applications, B-Trees of large order (e.g., m = 128) are more common than low-order B-Trees such as 2-3 trees.

Questions and Answers